

## Пояснение к коду: слияние k отсортированных списков (OCaml)

### 1. Краткое описание задачи

**Задача:** считать из файла JSON набор отсортированных списков целых чисел, объединить их в один отсортированный список и посчитать количество сравнений, выполненных при слиянии. Результат записать в файл merged.json.

### 2. Краткий обзор алгоритма (крупные шаги)

- 1) Прочитать JSON-файл, содержащий массив списков (lists.json).
- 2) Сливаем две отсортированные последовательности и считаем сравнения.
- 3) Рекурсивное разбиение списка списков: делим массив списков пополам, рекурсивно сливаем левые и правые половины, затем применяем merge\_two.
- 4) Записать итоговый отсортированный список и число сравнений в JSON-файл merged.json.

### 3. Пояснение импортов

open Yojson.Basic

open Yojson.Basic.Util

Пояснение:

- Yojson.Basic: модуль для чтения и записи JSON в OCaml (работа с ByteString-представлением).

- Yojson.Basic.Util: вспомогательные функции для извлечения значений из JSON (to\_list, to\_int и т.п.).

### 4. Объяснение каждого блока и функции (сверху вниз)

#### 4.1 Функция take

Код:

```
let rec take n l =  
  if n <= 0 then []  
  else match l with  
  | [] -> []  
  | h :: t -> h :: take (n - 1) t
```

Объяснение:

- Назначение: вернуть первые  $n$  элементов списка  $l$ . Если  $n \leq 0$  или список пуст, возвращается пустой список.

- Рекурсивная реализация: каждый шаг берёт голову  $h$  и рекурсивно берёт оставшиеся  $n-1$  элементов из хвоста  $t$ .

#### 4.2 Функция `drop`

Код:

```
let rec drop n l =  
  if n <= 0 then l  
  else match l with  
    | [] -> []  
    | h :: t -> drop (n - 1) t
```

Объяснение:

- Назначение: вернуть список после удаления первых  $n$  элементов. Если  $n \leq 0$  — вернуть исходный список; если список короче  $n$  — вернуть пустой список.

- Рекурсивно пропускаем голову до тех пор, пока не 'сбросим'  $n$  элементов.

#### 4.3 Функция `merge_two`

Код:

```
let rec merge_two a b cnt =  
  match a, b with  
  | [], b -> b, cnt  
  | a, [] -> a, cnt  
  | ha::ta, hb::tb ->  
    if ha <= hb then  
      let r, c = merge_two ta b (cnt + 1) in ha :: r, c  
    else  
      let r, c = merge_two a tb (cnt + 1) in hb :: r, c
```

Объяснение:

- Назначение: слияние двух отсортированных списков  $a$  и  $b$  в один отсортированный список; одновременно считаем количество сравнений ( $cnt$ ).

- Базовые случаи: если один из списков пуст, возвращаем другой список и текущий счётчик без изменений.

- Рекурсивный случай: сравниваем головные элементы  $ha$  и  $hb$  (это одно логическое сравнение).

- Если  $ha \leq hb$ , выбираем  $ha$  как следующий элемент результирующего списка, рекурсивно сливаем  $ta$  (хвост  $a$ ) с  $b$ , и увеличиваем счётчик сравнений на 1 ( $cnt + 1$ ).
- Иначе выбираем  $hb$  и рекурсивно сливаем  $a$  с  $tb$ , также увеличивая счётчик.
- Возвращаем пару:  $(merged\_list, updated\_count)$ .

#### 4.4 Функция `merge_k`

Код:

```
let rec merge_k lists cnt =
  match lists with
  | [] -> [], cnt
  | [x] -> x, cnt
  | _ ->
    let mid = List.length lists / 2 in
    let left = take mid lists in
    let right = drop mid lists in
    let merged_left, cnt1 = merge_k left cnt in
    let merged_right, cnt2 = merge_k right cnt1 in
    merge_two merged_left merged_right cnt2
```

Объяснение:

- Назначение: объединить произвольное число отсортированных списков ( $lists$ ) с подсчётом сравнений, используя стратегию «разделяй и властвуй».
- Базовые случаи:
  - Пустой список списков: возвращаем пустой список и текущий счётчик.
  - Один список в коллекции: возвращаем этот список и текущий счётчик (ничего не нужно сливать).
- Рекурсивный случай:
  - Находим  $mid = \text{длина} / 2$ ; разбиваем исходный массив списков на  $left$  (первые  $mid$ ) и  $right$  (остаток) с помощью функций `take` и `drop`.
  - Рекурсивно объединяем  $left$ , получаем  $merged\_left$  и обновлённый счётчик  $cnt1$ .
  - Затем объединяем  $right$ , передавая  $cnt1$ , получаем  $merged\_right$  и  $cnt2$ .
  - Наконец, сливаем  $merged\_left$  и  $merged\_right$  через `merge_two`, передавая текущий счётчик  $cnt2$ ; возвращаем итоговый список и финальный счётчик.

#### 4.5 Главный блок (выполнение скрипта)

Код (упрощённо):

```

let () =
  let json = Yojson.Basic.from_file "lists.json" in
  let lists = json |> to_list |> List.map (fun x -> x |> to_list |> List.map to_int) in
  let result, comparisons = merge_k lists 0 in

  let output = `Assoc [
    "result", `List (List.map (fun x -> `Int x) result);
    "comparisons", `Int comparisons
  ] in

  let json_str = Yojson.Basic.pretty_to_string output in
  let oc = open_out "merged.json" in
  output_string oc json_str;
  close_out oc;
  Printf.printf "Done! Comparisons: %d → merged.json
" comparisons

```

Объяснение:

- Чтение входа: `Yojson.Basic.from_file "lists.json"` читает JSON с массивом списков.
- Преобразование: выражение `json |> to_list |> List.map (fun x -> x |> to_list |> List.map to_int)` превращает JSON-массив списков в OCaml-список списков `int (int list list)`.
- Вызов алгоритма: `let result, comparisons = merge_k lists 0` — запускаем объединение с начальным счётчиком сравнений 0.
- Формирование выходного JSON: создаём объект с полями "result" (список ints) и "comparisons" (целое число).
- Сериализация и запись: `pretty_to_string` делает читаемую строку JSON; открываем файл `merged.json` и записываем туда результат.
- Печать в консоль: выводим количество сравнений по завершении.